Keras課堂實驗(三)

道瓊指數

高怡宣
陽明大學 生物醫學影像暨放射科學系

Data: DJIA 30 Stock Time Series
https://www.kaggle.com/szrlee/stock-time-series-20050101-to-20171231

Code: Experimental data analysis and prediction using LSTM
https://www.kaggle.com/shubhamgupta2208/eda-and-prediction-using-lstm

python

ANACONDA

Keras: The Python Deep Learning library

Keras

jupyter

TensorFlow

kaggle

kaggle | Search kaggle | Competitions | Datasets | Kernels | Discussion | Learn | •••

Dataset

DJIA 30 Stock Time Series

Historical stock data for DIJA 30 companies (2006-01-01 to 2018-01-01)

道瓊指數

szrlee • updated 10 months ago (Version 3)

Data | Overview | Kernels | Discussion | Activity | Download (3 MB) | New Kernel

Data (3 MB) | API | kaggle datasets download -d szrlee/stock-time-se… | ? | Download All

Data Sources

- AABA_2006-01-01_t... 3020 x 7
- AAPL_2006-01-01_t... 3020 x 7
- all_stocks_2006-01-... 93.6k x 7
- all_stocks_2017-01-0...

About this file | Edit

Help us describe this file

Columns | Edit

- Date
- Open
- High

https://www.kaggle.com/szrlee/stock-time-series-20050101-to-20171231

kaggle Search kaggle Competitions Datasets Kernels Disc

ShubhamGupta

EDA and prediction using LSTM

last run 7 months ago · IPython Notebook HTML · 668 views
using data from DJIA 30 Stock Time Series · Public

Notebook Code Data (1) Comments (3) Log Versions (1) Forks (27)

Notebook

Exploratory Data Analysis, EDA

Long Short Term Memory, LSTM

Stock Analysis

https://www.kaggle.com/shubhamgupta2208/eda-and-prediction-using-lstm

jupyter Keras_04_Stock Last Checkpoint: 幾秒前 (autosaved)

File Edit View Insert Cell Kernel Widgets Help

Run Markdown

Stock Analysis

data from: ***https://www.kaggle.com/szrlee/stock-time-series-20050101-to-20171231***

code modified from: ***https://www.kaggle.com/shubhamgupta2208/eda-and-prediction-using-lstm***

Table of Contents

1. Introduction
2. Load libraries and global variables
3. Read Data

Table of Contents

1. Introduction
2. Load libraries and global variables
3. Read Data
4. Data Overview
5. Data Cleaning
6. Feature Engineering
7. Time Series Analysis
8. Corelation Analysis
9. Data Modelling
10. Model Selection
11. Fetch Data
12. Feature Scaling
13. Train-test timestep data creation
14. Forecasting
15. Deep Neural Nets
16. Prediction Analysis
17. Conclusion

1. Exploratory Data Analysis, EDA
2. Long Short Term Memory, LSTM

Introduction

This notebook analyzes content present in the `DIJA 30 Stock Time Series` dataset. It provides detailed insight into the stock trends and also includes a guide to train and fit a LSTM model for stock price prediction.

Load libraries and set global options

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

In [2]:
```
pd.options.display.float_format = '{:.2f}'.format
sns.set(rc={'figure.figsize':(20, 20)})
```

In [3]:
```
import sys
print("Python version: {}". format(sys.version))

import pandas as pd
print("pandas version: {}". format(pd.__version__))

import matplotlib
print("matplotlib version: {}". format(matplotlib.__version__))

import numpy as np
print("NumPy version: {}". format(np.__version__))
```

```
Python version: 3.6.5 |Anaconda, Inc.| (default, Mar 29 2018, 13:32:41)
pandas version: 0.23.0
matplotlib version: 2.2.2
NumPy version: 1.14.3
SciPy version: 1.1.0
IPython version: 6.4.0
scikit-learn version: 0.19.1
```

```
C:\Users\USER\Anaconda3\lib\site-packages\h5py\__init__.py:36: FutureWar
argument of issubdtype from `float` to `np.floating` is deprecated. In 
np.float64 == np.dtype(float).type`.
  from .._conv import register_converters as _register_converters
Using TensorFlow backend.
```

```
keras version: 2.2.2
tensorflow version: 1.10.0
```

Read Data

In [4]:
```
df = pd.read_csv('../input/all_stocks_2006-01-01_to_2018-01-01.csv')
```

Data Overview

In [5]:
```
df.head()
```

Out[5]:

	Date	Open	High	Low	Close	Volume	Name
0	2006-01-03	77.76	79.35	77.24	79.11	3117200	MMM
1	2006-01-04	79.49	79.49	78.25	78.71	2558000	MMM
2	2006-01-05	78.41	78.65	77.56	77.99	2529500	MMM
3	2006-01-06	78.64	78.90	77.64	78.63	2479500	MMM

From the website, we find the following information about the columns:

- `Date` - Date for which the price is given
- `Open` - Price of the stock at market open (In USD)
- `High` - Highest price reached in the day
- `Low` - Lowest price reached in the day
- `Close` - Closing price for the day
- `Volume` - Number of shares traded `-` Name` - the stock's ticker name

Further, the author has mentioned that the data has been collected using the `pandas_datareader` package which fetches data from Google Finance API. cause for concern as the API has long been deprecated.

In [6]:
```
df = pd.read_csv('../input/all_stocks_2006-01-01_to_2018-01-01.csv',
```

In [7]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 93612 entries, 0 to 93611
Data columns (total 7 columns):
Date      93612 non-null datetime64[ns]
Open      93587 non-null float64
High      93602 non-null float64
Low       93592 non-null float64
Close     93612 non-null float64
Volume    93612 non-null int64
Name      93612 non-null object
dtypes: datetime64[ns](1), float64(4), int64(1), object(1)
memory usage: 5.0+ MB
```

In [8]:
```
df.Date = pd.to_datetime(df.Date)
```

In [9]:
```
df.describe()
```

Out[9]:

	Open	High	Low	Close	Volume
count	93587.00	93602.00	93592.00	93612.00	93612.00
mean	85.62	86.39	84.84	85.64	20156670.14
std	108.15	108.96	107.23	108.12	34421077.71
min	6.75	7.17	0.00	6.66	0.00
25%	33.95	34.29	33.60	33.96	5040180.50
50%	60.04	60.63	59.49	60.05	9701141.50
75%	94.00	94.74	93.25	94.01	20752221.50
max	1204.88	1213.41	1191.15	1195.83	843264044.00

The dataset has some missing values. We will analyze this and see how to fix it.

In [10]:
```
df.isnull().sum()
```

Out[10]:
```
Date       0
Open      25
High      10
Low       20
Close      0
Volume     0
Name       0
dtype: int64
```

The `Open` column has the maximum number of null values. Let's find the rows f
are missing.

In [11]:
```
df[df.Open.isnull()]
```

Out[11]:

	Date	Open	High	Low	Close	Volume	Name
2913	2017-07-31	nan	201.66	nan	201.17	1833625	MMM
5933	2017-07-31	nan	85.70	nan	85.23	3079797	AXP
11972	2017-07-31	nan	nan	nan	242.46	5777271	BA
14992	2017-07-31	nan	nan	nan	113.95	4486013	CAT
18012	2017-07-31	nan	110.00	nan	109.19	7561205	CVX
24051	2017-07-31	nan	nan	45.79	45.84	13622891	KO
25815	2012-08-01	nan	nan	nan	49.14	0	DIS
27071	2017-07-31	nan	110.14	nan	109.93	6815349	DIS

30091	2017-07-31	nan	80.39	nan	80.04	12820175	XOM
33111	2017-07-31	nan	25.69	nan	25.61	30616287	GE
36131	2017-07-31	nan	225.50	nan	225.33	1999637	GS
39151	2017-07-31	nan	149.98	nan	149.60	5253318	HD
42171	2017-07-31	nan	144.93	nan	144.67	4355718	IBM
48210	2017-07-31	nan	133.58	nan	132.72	5440788	JNJ
51230	2017-07-31	nan	92.36	nan	91.80	11520329	JPM
54250	2017-07-31	nan	nan	154.85	155.14	4280364	MCD
57269	2017-07-31	nan	nan	63.51	63.88	11714888	MRK
63308	2017-07-31	nan	59.12	58.33	59.05	6225907	NKE
66328	2017-07-31	nan	nan	nan	33.16	21982573	PFE

72368	2017-07-31	nan	nan	nan	128.09	1077258	TRV
75388	2017-07-31	nan	119.34	118.44	118.57	2172327	UTX
78408	2017-07-31	nan	nan	nan	191.81	3776476	UNH
81428	2017-07-31	nan	48.57	nan	48.40	28456827	VZ
84448	2017-07-31	nan	nan	nan	79.99	6263189	WMT

Interesting! The data is missing only for 31 July, 2017. This could be because:

- The API had an unexpected error while fetching the data.
- The data for this day does not exist in the source.

Let's check the number of `business days` for which the records as missing.

In [12]:
```
rng = pd.date_range(start='2006-01-01', end='2018-01-01', freq='B')
rng[~rng.isin(df.Date.unique())]
```

Out[12]:
```
DatetimeIndex(['2006-01-02', '2006-01-16', '2006-02-20', '2006-04-14',
               '2006-05-29', '2006-07-04', '2006-09-04', '2006-11-23',
               '2006-12-25', '2007-01-01',
               ...
               '2017-01-02', '2017-01-16', '2017-02-20', '2017-04-14',
               '2017-05-29', '2017-07-04', '2017-09-04', '2017-11-23',
               '2017-12-25', '2018-01-01'],
              dtype='datetime64[ns]', length=111, freq=None)
```

There are about 111 days for which the stock price data is missing. This could lead to potential problems with the analysis.

```
In [13]: df.groupby('Name').count().sort_values('Date', ascending=False)['Date']
```

```
Out[13]: Name
         JNJ       3020
         JPM       3020
         WMT       3020
         VZ        3020
         UTX       3020
         UNH       3020
         TRV       3020
         PG        3020
         PFE       3020
         NKE       3020
         MMM       3020
         MCD       3020
         CAT       3020
         DIS       3020
         CVX       3020
         CSCO      3019
         AMZN      3019
         INTC      3019
         AAPL      3019
         MSFT      3019
         MRK       3019
         GOOGL     3019
         AABA      3019
         Name: Date, dtype: int64
```

In [14]:
```
gdf = df[df.Name == 'AABA']
cdf = df[df.Name == 'CAT']
```

In [15]:
```
cdf[~cdf.Date.isin(gdf.Date)]
```

Out[15]:

	Date	Open	High	Low	Close	Volume	Name
13147	2010-04-01	63.29	64.20	63.29	63.99	7830447	CAT

Some of the companies(Google, Microsoft, etc.) don't have an entry for the date 2010-04-01.

Let's check if all the listed companies have an entry on each date.

In [16]:
```
# Total number of companies
df.Name.unique().size
```

Out[16]:
```
31
```

In [17]:
```
df.groupby('Date').Name.unique().apply(len)
```

Out[17]:
```
Date
2006-01-03    31
2006-01-04    31
2006-01-05    31
2006-01-06    31
2006-01-09    31
2006-01-10    31
2006-01-11    31
```

```
2017-12-14    31
2017-12-15    31
2017-12-18    31
2017-12-19    31
2017-12-20    31
2017-12-21    31
2017-12-22    31
2017-12-26    31
2017-12-27    31
2017-12-28    31
2017-12-29    31
Name: Name, Length: 3020, dtype: int64
```

This confirms that each company had a stock price entry on each day.

Data Cleaning

Let us first fill in the null values on date 31 july, 2017 with the values from the previous day(i.e 28th July, 2017)

In [18]:
```
df.set_index('Date', inplace=True)

#Backfill `Open` column
values = np.where(df['2017-07-31']['Open'].isnull(), df['2017-07-28']['Open'], df['2017-07-31']['Open']
df['2017-07-31']= df['2017-07-31'].assign(Open=values.tolist())

values = np.where(df['2017-07-31']['Close'].isnull(), df['2017-07-28']['Close'], df['2017-07-31']['Clos
df['2017-07-31']= df['2017-07-31'].assign(Close=values.tolist())

values = np.where(df['2017-07-31']['High'].isnull(), df['2017-07-28']['High'], df['2017-07-31']['High']
df['2017-07-31']= df['2017-07-31'].assign(High=values.tolist())

values = np.where(df['2017-07-31']['Low'].isnull(), df['2017-07-28']['Low'], df['2017-07-31']['Low'])
df['2017-07-31']= df['2017-07-31'].assign(Low=values.tolist())

df.reset_index(inplace=True)
```

In [19]:
```
df[df.Date == '2017-07-31']
```

Out[19]:

	Date	Open	High	Low	Close	Volume	Name
2913	2017-07-31	200.79	201.66	198.69	201.17	1833625	MMM
5933	2017-07-31	83.88	85.70	83.62	85.23	3079797	AXP
8952	2017-07-31	149.90	150.33	148.13	148.73	19845920	AAPL
11972	2017-07-31	240.82	242.00	238.55	242.46	5777271	BA
14992	2017-07-31	114.45	114.90	113.48	113.95	4486013	CAT
18012	2017-07-31	106.71	110.00	106.36	109.19	7561205	CVX
21031	2017-07-31	31.54	31.59	31.37	31.45	19256428	CSCO
24051	2017-07-31	46.00	46.12	45.79	45.84	13622891	KO
27071	2017-07-31	109.98	110.14	109.66	109.93	6815349	DIS

78408	2017-07-31	189.11	191.24	187.60	191.81	3776476	UNH
81428	2017-07-31	47.70	48.57	47.12	48.40	28456827	VZ
84448	2017-07-31	79.80	79.96	79.54	79.99	6263189	WMT
87467	2017-07-31	960.00	961.19	941.72	945.50	2293389	GOOGL
90486	2017-07-31	1019.05	1019.05	987.02	987.78	7352063	AMZN
93505	2017-07-31	59.70	59.89	58.07	58.40	6738130	AABA

We can confirm that the backfill has worked as expected.

Simlarly, we noticed that 8 of the 31 stocks have missing data on 1st April, 2014. As done before, we will use the stock prices of the previous day to fill the data.

```
missing_data_stocks = ['CSCO','AMZN','INTC','AAPL','MSFT','MRK','GOOGL', 'AABA']
```

```
columns = df.columns.values
```

```
for stock in missing_data_stocks:
    tdf = df[(df.Name == stock) & (df.Date == '2014-03-28')].copy()
    tdf.Date = '2014-04-01'
    pd.concat([df, tdf])
print("Complete")
```

Let's check if the backfill worked as expected.

In [23]:
```
df[(df.Name == 'CSCO') & (df.Date == '2014-04-01')]
```

Out[23]:

	Date	Open	High	Low	Close	Volume	Name
20192	2014-04-01	22.33	23.31	22.30	23.10	80113826	CSCO

Awesome! The backfill has worked for that particular day

Finally, there is just one more null record. We will drop that record.

In [24]:
```
df[df.Open.isnull()]
```

Out[24]:

	Date	Open	High	Low	Close	Volume	Name
25815	2012-08-01	nan	nan	nan	49.14	0	DIS

In [25]:
```
df = df[~((df.Date == '2012-08-01') & (df.Name == 'DIS'))]
```

Let's perform a quick sanity check for null values again.

In [26]:
```
df.isnull().sum()
```

Out[26]:
```
Date      0
Open      0
High      0
Low       0
Close     0
Volume    0
Name      0
dtype: int64
```

Feature Engineering

Since we have four values of stock price for each day, let's create a feature called `Price` which is the average of all these values.

```
values = (df['High'] + df['Low'] + df['Open'] + df['Close'])/4
df = df.assign(Price=values)
```

In [28]:
```
df.head()
```

Out[28]:

	Date	Open	High	Low	Close	Volume	Name	Price
0	2006-01-03	77.76	79.35	77.24	79.11	3117200	MMM	78.36
1	2006-01-04	79.49	79.49	78.25	78.71	2558000	MMM	78.98
2	2006-01-05	78.41	78.65	77.56	77.99	2529500	MMM	78.15
3	2006-01-06	78.64	78.90	77.64	78.63	2479500	MMM	78.45
4	2006-01-09	78.50	79.83	78.46	79.02	1845600	MMM	78.95

In [29]:
```
df.Price.describe()
```

Out[29]:
```
count    93611.00
mean        85.63
std        108.11
min          6.73
25%         33.95
50%         60.07
75%         94.00
max       1200.76
Name: Price, dtype: float64
```

We can see that 75% of the stocks have a price of under 94$,

Let's go one step further and compute the daily growth of the stock prices compared to day 1 of the prices(i.e compute cumalative compound growth)

```
stock_names = df.Name.unique()
```

```
day_prices = df[df.Date == df.Date.min()].Price
```

```
price_mapping = {n : c for n, c in zip(stock_names, day_prices)}
```

```
base_mapping = np.array(list(map(lambda x : price_mapping[x], df['Name'].values))
```

In [34]:
```
df['Growth'] = df['Price'] / base_mapping - 1
```

In [35]:
```
df.Growth.describe()
```

Out[35]:
```
count    93611.00
mean         0.79
std          1.91
min         -0.81
25%          0.04
50%          0.34
75%          0.82
max         24.39
Name: Growth, dtype: float64
```

Inferences

Wow! The worst performing company had a decline of 81% in their shares compared to their first ever opening price and the best company had a whopping 2439% increase in their share price. (*Hint*: EC2 instances)

Time Series Analysis

Let's find out the top 5 best and worst performing stocks!

```
sample_dates = pd.date_range(start='2006-01-01', end='2018-01-01', freq='B')
```

```
year_end_dates = sample_dates[sample_dates.is_year_end]
```

```
year_end_dates
```

```
DatetimeIndex(['2006-12-29', '2007-12-31', '2008-12-31', '2009-12-31',
               '2010-12-31', '2011-12-30', '2012-12-31', '2013-12-31',
               '2014-12-31', '2015-12-31', '2016-12-30', '2017-12-29'],
              dtype='datetime64[ns]', freq=None)
```

```
In [39]: worst_stocks = df[df.Date == df.Date.max()].sort_values('Growth')

In [40]: best_stocks = df[df.Date == df.Date.max()].sort_values('Growth',

In [41]: ws = worst_stocks.Name.values

In [42]: bs = best_stocks.Name.values

In [43]: tdf = df.copy()

In [44]: tdf = df.set_index('Date')
```

```
worst_stocks
```

	Date	Open	High	Low	Close	Volume	Name	Price	Growth
33217	2017-12-29	17.27	17.53	17.27	17.45	76149513	GE	17.38	-0.51
30197	2017-12-29	84.00	84.20	83.64	83.64	8523411	XOM	83.87	0.46
66434	2017-12-29	36.23	36.49	36.22	36.22	11421487	PFE	36.29	0.53
69454	2017-12-29	92.24	92.46	91.88	91.88	4346675	PG	92.11	0.58
93611	2017-12-29	69.79	70.13	69.43	69.85	6613070	AABA	69.80	0.74

```
tdf[tdf.Name.isin(ws)].groupby('Name').Growth.plot(\
        title='Historical trend of worst 5 stocks of 2017', legend=True)
```

```
Name
AABA    AxesSubplot(0.125,0.2;0.775x0.68)
GE      AxesSubplot(0.125,0.2;0.775x0.68)
PFE     AxesSubplot(0.125,0.2;0.775x0.68)
PG      AxesSubplot(0.125,0.2;0.775x0.68)
XOM     AxesSubplot(0.125,0.2;0.775x0.68)
```

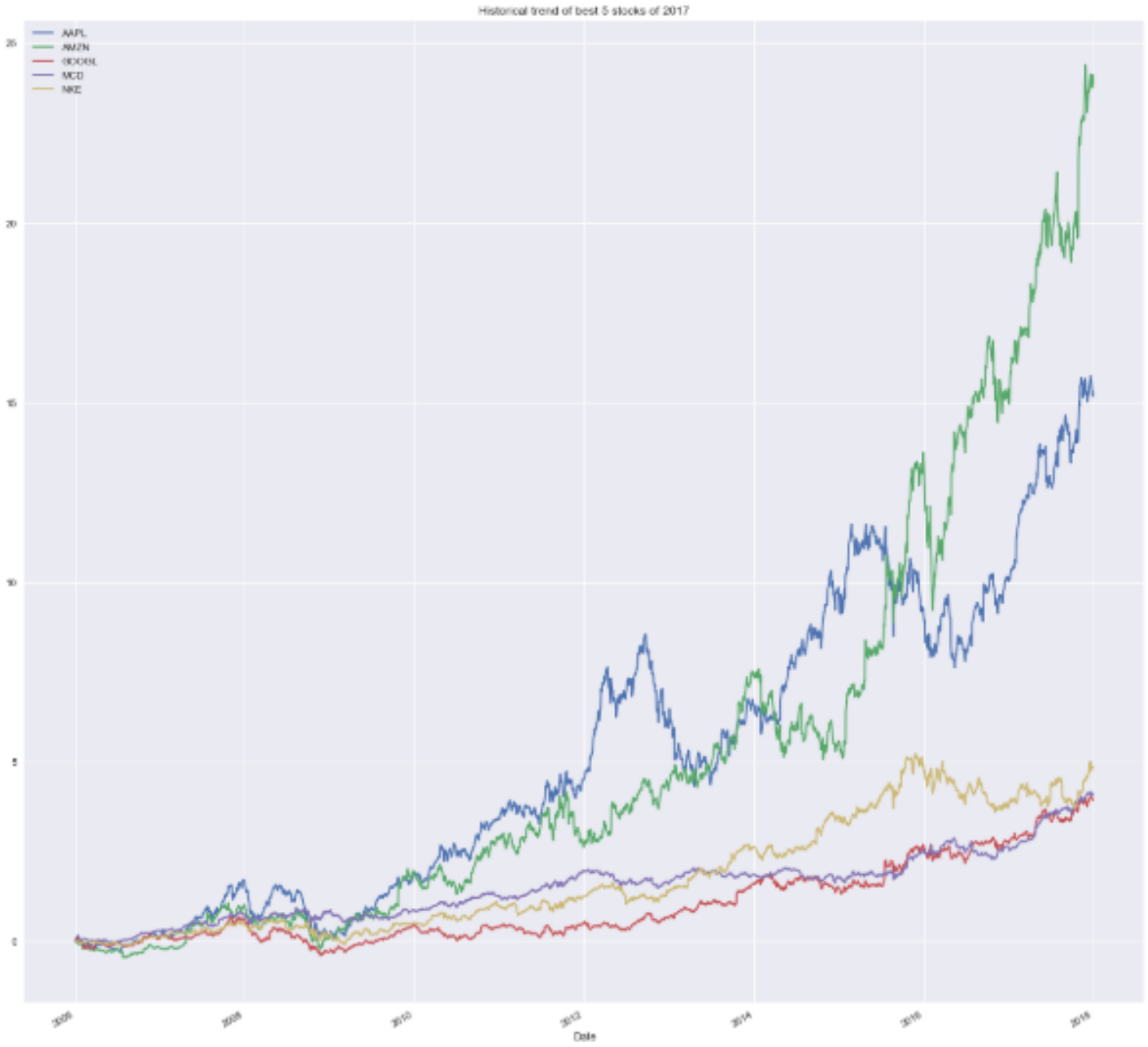
Historical trend of worst 5 stocks of 2017
AABA
GE
PFE
PG
BOM
0.8
0.6
0.4
0.2
0.0
-0.2
-0.4
-0.6
-0.8
2006
2008
2010
2012
2014
2016
2018
Date

```
best_stocks
```

	Date	Open	High	Low	Close	Volume	Name	Price	Growth
90592	2017-12-29	1182.35	1184.00	1167.50	1169.47	2688391	AMZN	1175.83	23.87
9058	2017-12-29	170.52	170.59	169.22	169.23	25999922	AAPL	169.89	15.17
63414	2017-12-29	63.21	63.37	62.55	62.55	5222115	NKE	62.92	4.83
54356	2017-12-29	172.98	172.98	171.92	172.12	2201771	MCD	172.50	4.10
87573	2017-12-29	1055.49	1058.05	1052.70	1053.40	1180340	GOOGL	1054.91	3.93

```
tdf[tdf.Name.isin(bs)].groupby('Name').Growth.plot(\
        title='Historical trend of best 5 stocks of 2017', legend=True)
```

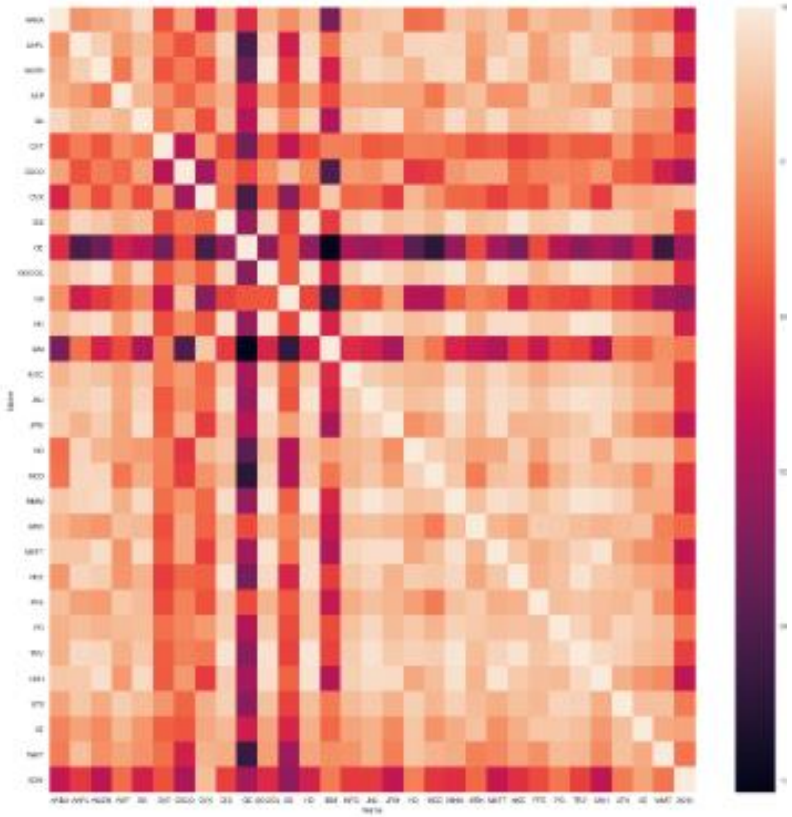
```
Name
AAPL      AxesSubplot(0.125,0.2;0.775x0.68)
AMZN      AxesSubplot(0.125,0.2;0.775x0.68)
GOOGL     AxesSubplot(0.125,0.2;0.775x0.68)
MCD       AxesSubplot(0.125,0.2;0.775x0.68)
NKE       AxesSubplot(0.125,0.2;0.775x0.68)
```


Historical trend of best 5 stocks of 2017
AAPL
AMZN
GOOGL
MCD
NKE
25
20
15
10
5
0
2006
2008
2010
2012
2014
2016
2018
Date

Corelation Analysis

Let us now try to find some corelation between the growth vs time of each stock in the dataset

In [49]:
```
corr = df.pivot('Date', 'Name', 'Growth').corr()
sns.heatmap(corr)
```



Let's us just focus on high positive and high negative correlations.

```
def unique_corelations(indices):
    mapping = {}
    for record in indices:
        (stock_a, stock_b) = record
        value_list = mapping.get(stock_a)
        if value_list:
            if stock_b not in value_list:
                value_list.append(stock_b)
                mapping.update({stock_a: value_list})
        else:
            mapping.update({stock_a: [stock_b]})

    return mapping
```

In [51]:
```
def filter_corelations_positive(corr, threshold=0.9):
    indices = np.where(corr > threshold)
    indices = [(corr.index[x], corr.columns[y]) for x, y in zip(*indices)
                                                if x != y and x < y]
    mapping = unique_corelations(indices)
    return mapping
```

In [52]:
```
filter_corelations_positive(corr, threshold=0.95)
```

Out[52]:
```
{'AMZN': ['GOOGL', 'HD', 'MSFT', 'UNH'],
 'BA': ['JPM'],
 'DIS': ['HD', 'NKE', 'TRV'],
 'GOOGL': ['HD', 'JNJ', 'MMM', 'MSFT', 'TRV', 'UNH'],
 'HD': ['JNJ', 'MMM', 'NKE', 'TRV', 'UNH'],
 'JNJ': ['MMM', 'MSFT', 'TRV'],
 'JPM': ['MSFT'],
 'MMM': ['MSFT', 'TRV', 'UNH'],
 'MSFT': ['UNH'],
 'NKE': ['TRV']}
```

```
def filter_corelations_positive(corr, threshold=0.9):
    indices = np.where(corr > threshold)
    indices = [(corr.index[x], corr.columns[y]) for x, y in zip(*indices)
                                                if x != y and x < y]
    mapping = unique_corelations(indices)
    return mapping
```

```
filter_corelations_negative(corr, -0.1)
```

```
{'GE': ['IBM', 'MCD']}
```

From the above results, we can note the following:-

- There is a **Strong Positive** corelation in the stock growth of GOOGL with MSFT, NKE,
- There is a **Weak Negative** corelation in the stock growth of GE with IBM and MCD.

1. Exploratory Data Analysis, EDA
2. Long Short Term Memory, LSTM

長短期記憶
（Long Short-Term Memory，LSTM）

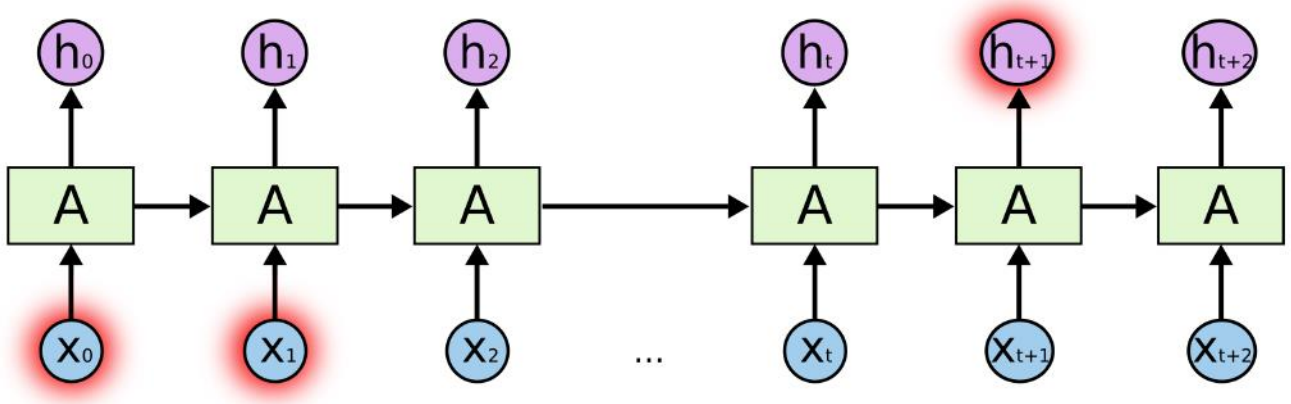
長短期記憶（Long Short Term Memory，LSTM）是一種時間遞歸神經網絡（Recurrent Neural Network，RNN），論文首次發表於1997年。LSTM的表現通常比一般的RNN更好。

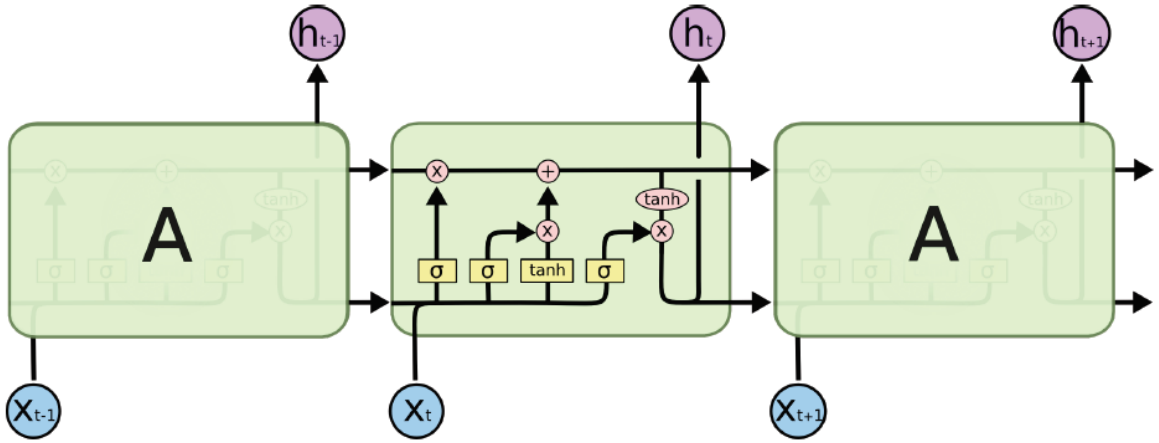
由於獨特的設計結構，**LSTM適合於處理時間序列中間隔和延遲非常長的重要事件，比如用在連續手寫識別上，以及語音識別。**

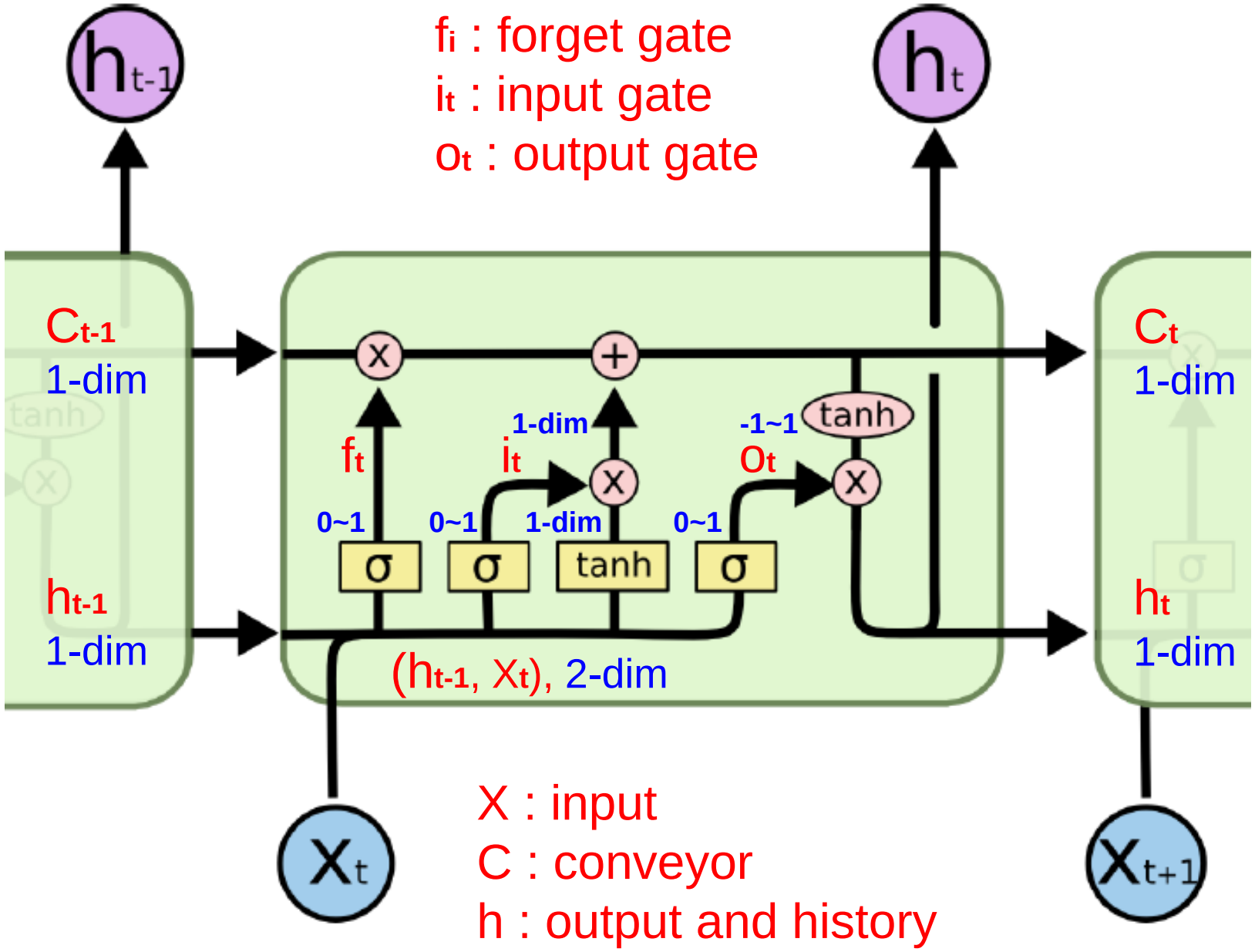
https://zh.wikipedia.org/wiki/%E9%95%B7%E7%9F%AD%E6%9C%9F%E8%A8%98%E6%86%B6

Understanding LSTM Networks

by Christopher Olah, posted on August 27, 2015
http://colah.github.io/posts/2015-08-Understanding-LSTMs/






h_{t-1}
h_t
f_i : forget gate
i_t : input gate
o_t : output gate
C_{t-1}
1-dim
h_{t-1}
1-dim
C_t
1-dim
h_t
1-dim
f_t
0~1
i_t
1-dim
0~1
1-dim
O_t
-1~1
0~1
tanh
σ
σ
tanh
σ
(h_{t-1}, X_t), 2-dim
X_t
X_{t+1}
X : input
C : conveyor
h : output and history

Let us try to forecast the prices of the Google Stock(GOOGL) and measure how close we came to the actual value.

Data Modelling

In [55]:
```
google_df = df[df.Name == 'GOOGL']
```

In [56]:
```
gdf = google_df[['Date', 'Price']].sort_values('Date')
```

Model Selection

As this is a time-series problem, we can use one of the following models to solve it:

- ARIMA/ARMA: Auto-Regressive Moving Average models are a class of model that captures a suite of different standard temporal structures in time series data.
- LSTM: Long-Short-Term_memory networks are a form of Recurrent Neural Networks. Few advantages of neural nets are:
  - Neural networks can model any non-linear function
  - Neural networks give good results without much parameter tuning

Hence, We will choose LSTM's as our model for forecasting stock prices.

We will try to predict the `Price` of the Google based on the previous 60 values(i.e stock prices on the previous 30 days)

Fetch Data

We will only use the `Price` column to forecast the future stock price for simplicity. all of the other columns.

In [57]:
```
training_set = gdf[gdf.Date.dt.year != 2017].Price.values
```

In [58]:
```
test_set =  gdf[gdf.Date.dt.year == 2017].Price.values
```

In [59]:
```
print("Training set size: ",training_set.size)
print("Test set size: ", test_set.size)
```

```
Training set size:  2768
Test set size:  251
```

Feature Scaling

As the amount of stock prices vary by a huge margin, we will scale the prices to be in the 0-1 range

In [60]:
```
from sklearn.preprocessing import MinMaxScaler
```

In [61]:
```
scaler = MinMaxScaler()
```

In [62]:
```
training_set_scaled = scaler.fit_transform(training_set.reshape(-1, 1))
```

Train and Test timestep data creation

For training, we will use previous 30 stock values to predict the stock price at time t. have to create our training and test set.

In [63]:
```
def create_train_data(training_set_scaled):
    X_train, y_train = [], []
    for i in range(30, training_set_scaled.size):
        X_train.append(training_set_scaled[i-30: i])
        y_train.append(training_set_scaled[i])
    # Converting list to numpy arrays
    X_train = np.array(X_train)
    y_train = np.array(y_train)
    return X_train, y_train
```

In [64]:
```
X_train, y_train = create_train_data(training_set_scaled)
```

Similarly, we'll create our test data set.

In [65]:
```
def create_test_data():
    X_test = []
    inputs = gdf[len(gdf) - len(test_set) - 30:].Price.values
    inputs = scaler.transform(inputs.reshape(-1, 1))
    for i in range(30, test_set.size+30): # Range of the number of valu
        X_test.append(inputs[i - 30: i, 0])
    X_test = np.array(X_test)
    X_test = np.reshape(X_test, (X_test.shape[0], X_test.shape[1], 1))
    return X_test
```

In [66]:
```
X_test = create_test_data()
```

In [67]:
```
X_test.shape
```

Out[67]:
```
(251, 30, 1)
```

Forecasting

Let us first start with a very simple model. We will use a single LSTM layer.

In [68]:
```
from keras.models import Sequential
from keras.layers import Dense, LSTM
```

In [69]:
```
def create_simple_model():
    model = Sequential()
    model.add(LSTM(units = 10, return_sequences = False, \
                   input_shape = (X_train.shape[1], 1)))
    model.add(Dense(units = 1))
    return model
```

We now need to pick the optimizer for our model and a function to measure how well the model is doing i.e loss. We will pick RMSE as the loss function and Sigmoid Gradient Descent(SGD) as our optimizer with default learning rate(i.e 0.01)

```
def compile_and_run(model, epochs=50, batch_size=64):
    model.compile(metrics=['accuracy'], optimizer='adam', loss='mean_squared_erro
    history = model.fit(X_train, y_train, epochs=epochs, batch_size=batch_size, v
    return history
```

```
def plot_metrics(history):
    metrics_df = pd.DataFrame(data={"loss": history.history['loss']})
    metrics_df.plot()
```

n = input = 1, h = neuron = 10
4 x (h^2+ h x n + h) =
4 x (100 + 10 + 10) = 480

```
In [72]: simple_model = create_simple_model()
         simple_model.summary()
         history = compile_and_run(simple_model, epochs=20)
```

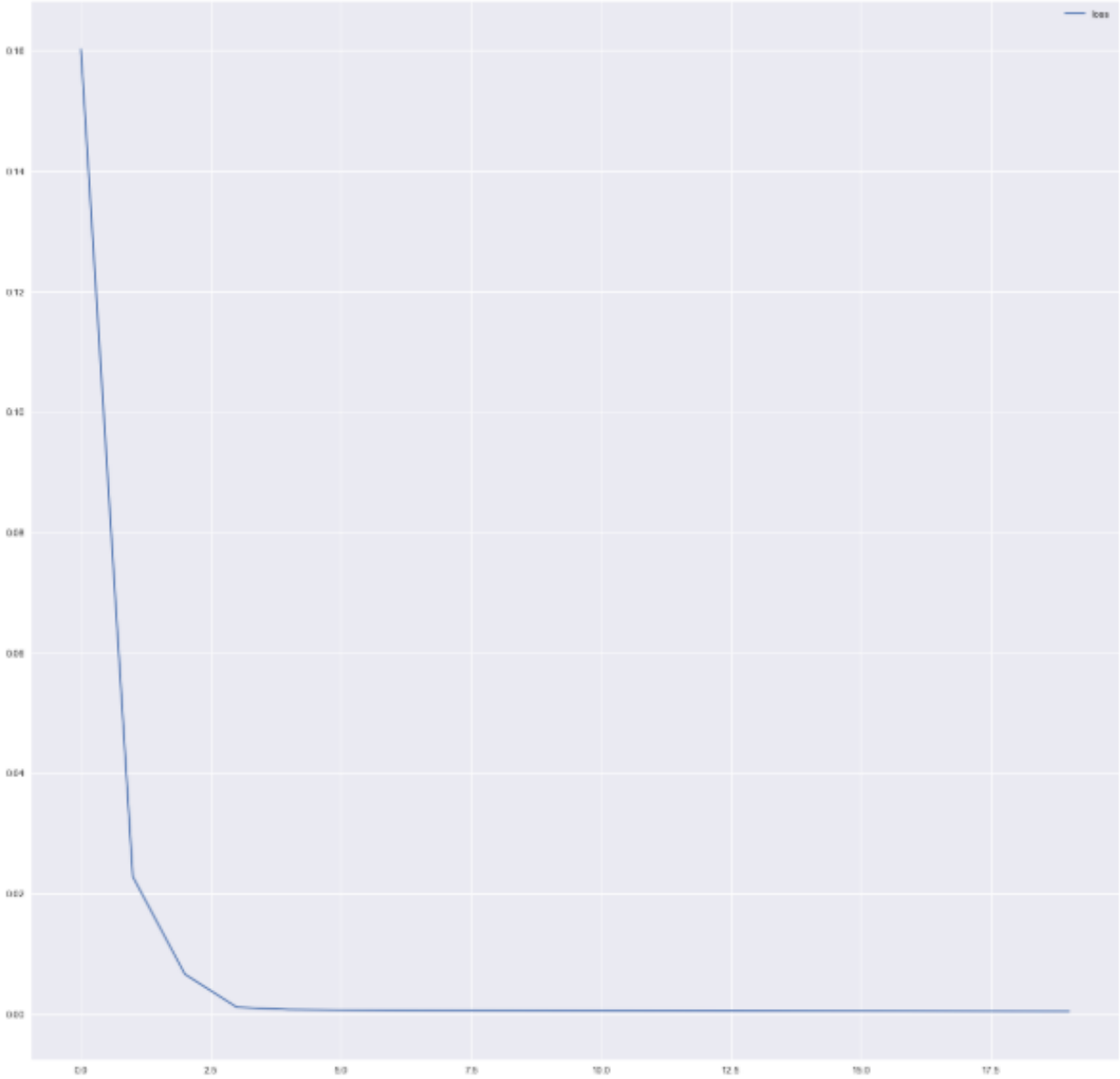
```
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
lstm_1 (LSTM)                (None, 10)                480
_________________________________________________________________
dense_1 (Dense)              (None, 1)                 11
=================================================================
Total params: 491
Trainable params: 491
Non-trainable params: 0
```

i = input = 10, c = neuron = 1
(i x c + 1) = (10 + 1) = 11

In [72]:
```
simple_model = create_simple_model()
simple_model.summary()
history = compile_and_run(simple_model, epochs=20)
```

```
=================================================================
Total params: 491
Trainable params: 491
Non-trainable params: 0
_________________________________________________________________
Epoch 1/20
Epoch 2/20
Epoch 3/20
Epoch 4/20
Epoch 5/20
Epoch 6/20
Epoch 7/20
Epoch 8/20
Epoch 9/20
Epoch 10/20
```

```
In [80]: plot_metrics(history)
```



Let us now use the model on the test set.

In [74]:
```
def make_predictions(X_test, model):
    y_pred = model.predict(X_test)
    final_predictions = scaler.inverse_transform(y_pred)
    fp = np.ndarray.flatten(final_predictions)
    ap = np.ndarray.flatten(test_set)
    pdf = pd.DataFrame(data={'Actual': ap, 'Predicted': fp})
    ax = pdf.plot()
```

```
In [75]: make_predictions(X_test, simple_model)
```

As we can see from the image above, although our model seems to have found some trend in the prices, it is *very* far away from the actual stock value. Hence, this model cannot be used in a production environment.

Question: Can we improve this model?

Lets find out.

Deep Neural Nets

As always, here is the solution to our problem

Deep neural nets can capture trends over a largely spread dataset and could improve our model. For this problem of forecasting, we will use a stacked LSTM(i.e multiple LSTM layers instead of 1). Further, we will also increase the number of units per LSTM cell to 50.

In [76]:
```
def create_dl_model():
    model = Sequential()

    # Adding the first LSTM layer
    model.add(LSTM(units = 50, return_sequences = True, \
                   input_shape = (X_train.shape[1], 1)))
    # Adding a second LSTM layer
    model.add(LSTM(units = 50, return_sequences = True))
    # Adding a third LSTM layer
    model.add(LSTM(units = 50, return_sequences = True))
    # Adding a fourth LSTM layer
    model.add(LSTM(units = 50))

    # Adding the output layer
    model.add(Dense(units = 1))
    return model
```

```
In [77]: dl_model = create_dl_model()
```

```
In [78]: dl_model.summary()
```

```
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
lstm_2 (LSTM)                (None, 30, 50)            10400
_________________________________________________________________
lstm_3 (LSTM)                (None, 30, 50)            20200
_________________________________________________________________
lstm_4 (LSTM)                (None, 30, 50)            20200
_________________________________________________________________
lstm_5 (LSTM)                (None, 50)                20200
_________________________________________________________________
dense_2 (Dense)              (None, 1)                 51
=================================================================
Total params: 71,051
Trainable params: 71,051
Non-trainable params: 0
```

lstm_2 :

n = input = 1, h = neuron = 50

$4 \times (h^2 + h \times n + h)$

$4 \times (2500 + 50 \times 1 + 50) = 10400$

lstm_3~5 :

n = input = 50, h = neuron = 50

$4 \times (h^2 + h \times n + h)$

$4 \times (2500 + 50 \times 50 + 50) = 20200$

dense_2 :

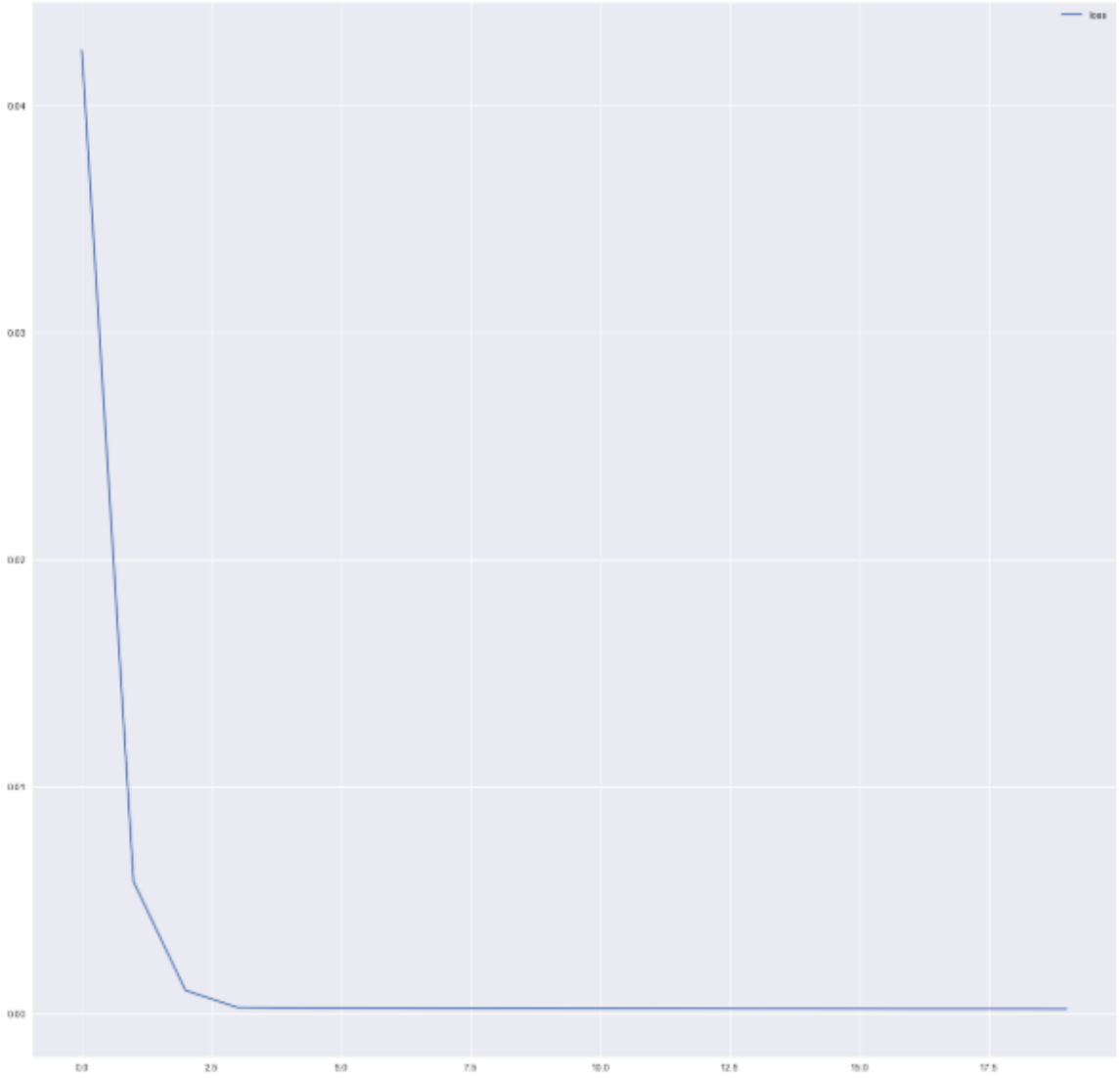
i = input = 50, c = neuron = 1

$(i \times c + 1) = 50 \times 1 + 1 = 51$

```
In [79]: history = compile_and_run(dl_model, epochs=20)
```

```
Epoch 1/20
Epoch 2/20
Epoch 3/20
Epoch 4/20
Epoch 5/20
Epoch 6/20
Epoch 7/20
Epoch 8/20
Epoch 9/20
Epoch 10/20
Epoch 11/20
Epoch 12/20
Epoch 13/20
Epoch 14/20
Epoch 15/20
Epoch 16/20
```

```
In [80]: plot_metrics(history)
```



Prediction Analysis

Let us try our deep neural net on the test set.

In [81]:
```
make_predictions(X_test, dl_model)
```

Actual
Predicted

As we can see, the deep neural net is a significant boost from our simple model. It has determined that the stock price is indeed related to the previous stock prices, and surprisngly, it is very accurate, so much so that at various points it is actually equal to the exact value of the stock on that day!

We could further improve this model by:

- **Deeper network** - Deep Neural Networks can learn relationships between data points seperated by a large time frame.
- **Dropout** - Adding a dropout layer would help prevent overfitting and stabalize the loss curves, which could give a better generalized model.

- **Training for longer** - Deep neural nets work better when they are trained for longer tend to work better. However, we must be careful not to overfit.
- **Adaptive Optimizers** - We could also use the Adam optimizer instead of SGD, which incorporates adaptive learning rates based on momentum. Adaptive optimizers should work better than SGD for this problem.
- **Hyper-parameter tuning** - Tuning the hyper-parameters is one of the simplest ways in which we can improve the network. We could use SGD with Restarts(part of the Fast.ai library) to help find the optimal learning rate, experiment with the number of units in each LSTM layer, etc.

Conclusion

In this notebook, we have acheived the following:

- Successfully analyzed the trends present in stock market prices of various companies and concluded that there is indeed a temporal relationship between these prices.
- Devised a model capable of reliably forecasting the stock prices of a company, thereby providing a tool to maximize profits.
- Discussed possible implementation details to deploy such a model at scale and make it available to a larger number of people.

Thank you for your attention